

```
#!/usr/bin/env python3
# inverted_hash_512x512_analysis.py
# 512×512 Inverted Hash - Complete Statistical Analysis
# Author: Marco Lazcano
# Fix applied: chunk=256 bytes (Option A) + 24 rounds (Option B)
# Input: 256 bytes | Output: 256 bytes (2048 bits) | No shortcuts.
```

```
import secrets
import time
import math
from collections import Counter
```

```
# =====
# S-BOX (AES - cryptographically strong)
# =====
SBOX = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab,
    0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72,
    0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31,
    0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2,
    0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f,
    0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58,
    0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f,
    0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3,
    0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19,
    0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b,
    0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4,
    0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae,
    0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b,
    0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d,
    0x9e,
```

```

    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28,
    0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb,
    0x16
]

```

```

ROUND_CONSTS = [
    0x243f6a88, 0x85a308d3, 0x13198a2e, 0x03707344, 0xa4093822, 0x299f31d0,
    0x082efa98, 0xec4e6c89, 0x452821e6, 0x38d01377, 0xbe5466cf, 0x34e90c6c,
    0xc0ac29b7, 0xc97c50dd, 0x3f84d5b5, 0xb5470917
]

```

```

# =====
# CORE FUNCTIONS
# =====

```

```

def rotate_left(x, n, bits=8):
    x = x & 0xFF
    n = n & 0x07
    return ((x << n) | (x >> (bits - n))) & 0xFF

```

```

def safe_byte(x):
    return x & 0xFF

```

```

def sacred_pair(x):
    """
    Sacred pair inversion: x -> bitwise complement (~x & 0xFF).
    Defines the mathematical boundary K1 / K2 at dimension 257.
    With 256-byte inputs the pyramid produces 255 anchors (i=0..254),
    so i >= 256 is a structural boundary reserved for future extensions.
    """
    return (~x) & 0xFF

```

```

def pyramidal_nonlinear(seq):
    """Build pyramid levels from a full 256-byte sequence."""
    temp = [safe_byte(b) for b in seq]
    levels = [temp[:]]
    while len(temp) > 1:
        next_level = []
        anchor = temp[0]
        for i in range(len(temp) - 1):
            diff = safe_byte(temp[i+1] - temp[i])
            diff = SBOX[diff]
            diff = safe_byte(diff ^ anchor)

```

```

        diff = rotate_left(diff, (i % 7) + 1)
        next_level.append(diff)
    if len(next_level) > 1:
        first = next_level.pop(0)
        next_level.append(first)
    levels.append(next_level)
    temp = next_level
return levels

```

```

def invert_pyramid(levels):
    """
    Invert pyramid: tip + reversed anchors.
    Sacred pair applied at i >= 256 (K1->K2 boundary).
    """
    if not levels:
        return []
    tip = levels[-1][0] if levels[-1] else 0
    left_anchors = [level[0] for level in levels[:-1] if level]
    inverted = [
        sacred_pair(a) if i >= 256 else a
        for i, a in enumerate(left_anchors)
    ]
    new_base = [tip] + inverted[::-1]
    while len(new_base) < 256:
        new_base.append(safe_byte(tip ^ len(new_base)))
    return new_base[:256]

```

```

# =====
# INVERTED HASH CLASS
# Fix A: full 256-byte pyramidal block per round
# Fix B: 24 rounds for complete diffusion
# =====

```

```

class InvertedHash:
    """
    512x512 Inverted Hash — v2 (corrected)
    State: 256 bytes (2048 bits)
    Output: 256 bytes (2048 bits)
    Chunk: 256 bytes — full state as one pyramidal block [Fix A]
    Rounds: 24 [Fix B]
    """

```

ROUNDS = 24 # Fix B: increased from 12 to 24

```

def __init__(self):
    self.reset()

def reset(self):
    self.state = bytes(256)
    self.buffer = bytearray()
    self.total_len = 0

def update(self, data):
    if isinstance(data, str):
        data = data.encode('utf-8')
    self.buffer.extend(data)
    self.total_len += len(data)
    while len(self.buffer) >= 256:
        block = self.buffer[:256]
        self.buffer = self.buffer[256:]
        self._absorb(block)

def _absorb(self, block):
    temp_state = bytearray(self.state)
    for i in range(256):
        temp_state[i] ^= block[i % len(block)]
    self.state = self._permutation(bytes(temp_state))

def _permutation(self, state):
    """
    Permutation with Fix A + Fix B:
    - Fix A: entire 256-byte state processed as ONE pyramidal block
    - Fix B: 24 rounds
    """
    state_vec = bytearray(state)
    for r in range(self.ROUNDS):

        # Fix A: one single 256-byte pyramidal inversion
        levels = pyramidal_nonlinear(list(state_vec))
        inverted_base = invert_pyramid(levels)
        for j in range(256):
            rv = safe_byte(ROUND_CONSTS[r % 16] >> ((j % 4) * 8))
            state_vec[j] ^= safe_byte(inverted_base[j] ^ rv)

        # Linear diffusion across full state
        for i in range(len(state_vec) - 1):
            state_vec[i] ^= state_vec[i + 1]

```

```

        # Nonlinear substitution
        for i in range(len(state_vec)):
            state_vec[i] = SBOX[state_vec[i]]

    return bytes(state_vec)

def digest(self, output_bytes=256):
    """Default output: 256 bytes (2048 bits)"""
    padding = bytearray()
    padding.append(0x80)
    while (len(self.buffer) + len(padding)) % 256 != 248:
        padding.append(0x00)
    padding.extend((self.total_len * 8).to_bytes(8, 'little'))
    self.buffer.extend(padding)
    while len(self.buffer) >= 256:
        block = self.buffer[:256]
        self.buffer = self.buffer[256:]
        self._absorb(block)
    output = bytearray()
    while len(output) < output_bytes:
        take = min(output_bytes - len(output), 256)
        output.extend(self.state[:take])
        self.state = self._permutation(self.state)
    return bytes(output[:output_bytes])

def hexdigest(self, output_bytes=256):
    return self.digest(output_bytes).hex()

# =====
# HELPER
# =====

def ih(msg, out=256):
    h = InvertedHash()
    h.update(msg if isinstance(msg, bytes) else msg.encode())
    return h.hexdigest(out)

# =====
# TEST 1 — BASIC FUNCTIONALITY & SENSITIVITY
# =====

def test_basic():

```

```

print("\n" + "="*65)
print("TEST 1: BASIC FUNCTIONALITY & SENSITIVITY")
print("="*65)
print(" Output: 256 bytes (2048 bits) — first 32 hex chars shown\n")

```

```

cases = [
    ("b" (empty)",    b""),
    ("b\\x00",        b"\\x00"),
    ("b\\x01",        b"\\x01"),
    ("b\\xff",        b"\\xff"),
    ("Hola Mundo",    b"Hola Mundo"),
    ("Hello",         b"Hello"),
    ("hello",         b"hello"),
    ("hello ' (space)", b"hello "),

```

```

]
hashes = []
for name, msg in cases:
    d = ih(msg)
    hashes.append(d)
    print(f" {name:<22} -> {d[:32]}...")

```

```

unique = len(set(hashes)) == len(hashes)
print(f"\n All hashes distinct: {'YES ✓' if unique else 'NO ✗'}")
return unique

```

```

# =====
# TEST 2 — DETERMINISM
# =====

```

```

def test_determinism(trials=200):
    print("\n" + "="*65)
    print(f"TEST 2: DETERMINISM ({trials} trials)")
    print("="*65)

```

```

errors = 0
for _ in range(trials):
    msg = secrets.token_bytes(256)
    h1 = InvertedHash(); h1.update(msg); d1 = h1.digest(256)
    h2 = InvertedHash(); h2.update(msg); d2 = h2.digest(256)
    if d1 != d2:
        errors += 1

```

```

print(f" Errors: {errors} / {trials}")

```

```
print(f" Result: {'PASS ✓' if errors == 0 else 'FAIL ✗'})
return errors == 0
```

```
# =====
# TEST 3 — AVALANCHE EFFECT (full 256-byte / 2048-bit output)
# =====
```

```
def test_avalanche(samples=150):
    print("\n" + "="*65)
    print(f"TEST 3: AVALANCHE EFFECT — FULL 2048-BIT OUTPUT ({samples} samples)")
    print("="*65)
    print(" Fix A+B applied: chunk=256 bytes, 24 rounds\n")
```

```
    avalanches = []
    for _ in range(samples):
        msg = secrets.token_bytes(256)
        pos = secrets.randbelow(256)
        bit = secrets.randbelow(8)
        mod = bytearray(msg)
        mod[pos] ^= (1 << bit)

        h1 = InvertedHash(); h1.update(msg); d1 = h1.digest(256)
        h2 = InvertedHash(); h2.update(bytes(mod)); d2 = h2.digest(256)

        diff = sum(bin(a ^ b).count('1') for a, b in zip(d1, d2))
        avalanches.append(diff / (256 * 8) * 100)
```

```
    mean = sum(avalanches) / len(avalanches)
    std = math.sqrt(sum((x - mean)**2 for x in avalanches) / len(avalanches))
```

```
    print(f" Mean:      {mean:.4f}% (ideal: 50.0000%)")
    print(f" Std Dev:    {std:.4f}% (ideal: < 5%)")
    print(f" Minimum:    {min(avalanches):.2f}%")
    print(f" Maximum:    {max(avalanches):.2f}%")
    print(f"\n Previous (v1): 30.94% std=13.63% △")
    print(f" Current (v2): {mean:.3f}% std={std:.3f}%")
```

```
    grade = "EXCELLENT ✓" if abs(mean-50) < 1 and std < 5 else \
            "GOOD ✓" if abs(mean-50) < 2 else "CHECK △"
    print(f" Result: {grade}")
    return mean, std
```

```
# =====  
# TEST 4 — SHANNON ENTROPY  
# =====
```

```
def test_entropy(samples=300):  
    print("\n" + "="*65)  
    print(f"TEST 4: SHANNON ENTROPY ({samples} samples, 256-byte output)")  
    print("="*65)  
  
    all_bytes = []  
    for _ in range(samples):  
        msg = secrets.token_bytes(256)  
        h = InvertedHash(); h.update(msg)  
        all_bytes.extend(h.digest(256))  
  
    freq = Counter(all_bytes)  
    total = len(all_bytes)  
    entropy = -sum((c/total) * math.log2(c/total) for c in freq.values())  
    eff = entropy / 8 * 100  
  
    print(f" Total bytes analyzed: {total:,}")  
    print(f" Unique byte values: {len(freq)} / 256")  
    print(f" Entropy: {entropy:.6f} bits/byte (maximum: 8.0)")  
    print(f" Efficiency: {eff:.4f}%")  
  
    grade = "EXCELLENT ✓" if entropy > 7.98 else \  
        "GOOD ✓" if entropy > 7.95 else "CHECK ⚠"  
    print(f" Result: {grade}")  
    return entropy
```

```
# =====  
# TEST 5 — CHI-SQUARE UNIFORMITY  
# =====
```

```
def test_chi_square(samples=600):  
    print("\n" + "="*65)  
    print(f"TEST 5: CHI-SQUARE UNIFORMITY ({samples} samples)")  
    print("="*65)  
  
    all_bytes = []  
    for _ in range(samples):  
        msg = secrets.token_bytes(256)  
        h = InvertedHash(); h.update(msg)
```



```

    all_bytes.extend(h.digest(256))

freq    = Counter(all_bytes)
total   = len(all_bytes)
expected = total / 256
chi2     = sum((freq.get(b, 0) - expected)**2 / expected for b in range(256))

print(f" Total bytes:      {total:.}")
print(f" Expected per value: {expected:.1f}")
print(f" Chi-square:        {chi2:.2f}")
print(f" Critical value 95%:  293.2 (255 degrees of freedom)")
print(f" Critical value 99%:  310.4")

grade = "EXCELLENT ✓" if chi2 < 293.2 else \
        "ACCEPTABLE ✓" if chi2 < 310.4 else "FAIL      ✗"
print(f" Result:   {grade}")
return chi2

# =====
# TEST 6 — PEARSON CORRELATION (full 256-byte output)
# =====

def test_correlation(samples=1000):
    n_bytes    = 256
    total_pairs = n_bytes * (n_bytes - 1) // 2 # 32,640

    print("\n" + "="*65)
    print(f"TEST 6: PEARSON CORRELATION — FULL 256-BYTE OUTPUT")
    print("="*65)
    print(f" Samples:      {samples} (N=1000 for statistical confidence)")
    print(f" Byte pairs:    {total_pairs:.}")
    print(f" Pairs/samples ratio: {total_pairs/samples:.1f} (was 81.6 at N=400)")
    print(f" Estimated time:  ~4-5 minutes\n")

    vals = [[] for _ in range(n_bytes)]
    t0 = time.time()

    for n in range(samples):
        msg = secrets.token_bytes(256)
        h = InvertedHash(); h.update(msg)
        d = h.digest(n_bytes)
        for i in range(n_bytes):
            vals[i].append(d[i])

```

```

if (n + 1) % 100 == 0:
    print(f" Processed {n+1}/{samples}...")

high_count = 0
counted    = 0
sum_r      = 0.0
max_r      = 0.0

for i in range(n_bytes):
    for j in range(i + 1, n_bytes):
        xi = vals[i]; xj = vals[j]
        mi = sum(xi)/samples; mj = sum(xj)/samples
        num = sum((a-mi)*(b-mj) for a,b in zip(xi,xj))
        di = math.sqrt(sum((a-mi)**2 for a in xi))
        dj = math.sqrt(sum((b-mj)**2 for b in xj))
        if di * dj > 0:
            r      = abs(num / (di * dj))
            sum_r  += r
            counted += 1
            if r > max_r: max_r = r
            if r > 0.10: high_count += 1

avg_r = sum_r / counted
elapsed = time.time() - t0

print(f"\n Time:           {elapsed:.1f} seconds")
print(f" Pairs analyzed:      {counted:,}")
print(f" Average correlation:    {avg_r:.5f} (ideal: ~0.000)")
print(f" Maximum correlation:    {max_r:.5f}")
print(f" Pairs with corr > 0.10: {high_count:,} of {counted:,}
({high_count/counted*100:.2f}%)")
print(f"\n NOTE: With N={samples} samples and {total_pairs:,} pairs,")
print(f" the pairs-to-samples ratio is {total_pairs/samples:.1f}.")
print(f" At N=1000 the statistical confidence is high enough to")
print(f" distinguish sampling variance from structural correlation.")
print(f" Average below 0.05 confirms genuine independence.")

grade = "EXCELLENT ✓" if avg_r < 0.05 else \
"GOOD ✓" if avg_r < 0.10 else \
"ACCEPTABLE ✓" if avg_r < 0.20 else "CHECK ⚠"
print(f" Result: {grade}")
return avg_r, max_r, high_count

```

```
# =====
# TEST 7 — S-BOX DIFFERENTIAL ANALYSIS
# =====
```

```
def test_sbox_differential():
    print("\n" + "="*65)
    print("TEST 7: S-BOX DIFFERENTIAL ANALYSIS")
    print("="*65)

    diff_table = [[0]*256 for _ in range(256)]
    for d in range(256):
        for x in range(256):
            diff_table[d][SBOX[x] ^ SBOX[x ^ d]] += 1

    max_prob = max(
        diff_table[d][o] / 256
        for d in range(1, 256)
        for o in range(256)
    )

    print(f" Max differential probability: {max_prob:.4f} ({max_prob*100:.2f}%)")
    print(f" Security threshold:      0.0625 (6.25%)")
    print(f" Result:    {'SECURE ✓' if max_prob <= 0.0625 else 'WEAK ✗'}")
    return max_prob
```

```
# =====
# TEST 8 — STRICT AVALANCHE CRITERION (SAC)
# =====
```

```
def test_sac(samples=150):
    """
    Strict Avalanche Criterion — full test.
    150 samples x 8 input bits x 2048 output bits.
    Estimated time: ~14 minutes on pure Python.
    Most rigorous and statistically reliable result.
    """

    print("\n" + "="*65)
    print(f"TEST 8: STRICT AVALANCHE CRITERION — SAC ({samples} samples)")
    print("="*65)
    print(" Fix A+B: each input bit should change each output bit ~50%")
    print(" Testing 8 input bits x 2048 output bits (full 256-byte output)")
    print(f" Estimated time: ~14 minutes on pure Python — worth the wait\n")
```

```

n_bits_in = 8
n_bits_out = 2048
sac_counts = [[0] * n_bits_out for _ in range(n_bits_in)]
t0 = time.time()

for s in range(samples):
    msg = secrets.token_bytes(256)
    h1 = InvertedHash(); h1.update(msg); d1 = h1.digest(256)

    for bit_in in range(n_bits_in):
        mod = bytearray(msg)
        mod[0] ^= (1 << bit_in)
        h2 = InvertedHash(); h2.update(bytes(mod)); d2 = h2.digest(256)

        for byte_out in range(256):
            xor_byte = d1[byte_out] ^ d2[byte_out]
            for bit_out in range(8):
                if (xor_byte >> bit_out) & 1:
                    sac_counts[bit_in][byte_out * 8 + bit_out] += 1

    if (s + 1) % 25 == 0:
        elapsed = time.time() - t0
        remaining = (elapsed / (s+1)) * (samples - s - 1)
        print(f" Processed {s+1}/{samples}... ({elapsed:.0f}s elapsed, ~{remaining:.0f}s
remaining)")

deviations = [
    abs(sac_counts[i][j] / samples - 0.5)
    for i in range(n_bits_in)
    for j in range(n_bits_out)
    if abs(sac_counts[i][j] / samples - 0.5) > 0.10
]

total_cells = n_bits_in * n_bits_out
pct = len(deviations) / total_cells * 100
elapsed = time.time() - t0

print(f"\n Time: {elapsed:.1f} seconds")
print(f" Cells analyzed: {total_cells:,}")
print(f" Cells with deviation >10%: {len(deviations)} of {total_cells} ({pct:.2f}%)")
print(f"\n Previous (v1, chunk=64, 12r): 75.27% △")
print(f" Current (v2, chunk=256, 24r): {pct:.2f}%")

grade = "EXCELLENT ✓" if pct < 5 else \

```

```

        "GOOD    ✓" if pct < 10 else "CHECK    △"
print(f" Result:    {grade}")
return pct

```

```

# =====
# TEST 9 — COLLISION RESISTANCE
# =====

```

```

def test_collisions(attempts=400):
    print("\n" + "="*65)
    print(f"TEST 9: COLLISION RESISTANCE  ({attempts} attempts)")
    print("="*65)

    seen    = {}
    collisions = 0
    t0      = time.time()

    for i in range(attempts):
        msg = secrets.token_bytes(256)
        h   = InvertedHash(); h.update(msg)
        sig = h.digest(8)

        if sig in seen:
            h1 = InvertedHash(); h1.update(seen[sig]); d1 = h1.digest(32)
            h2 = InvertedHash(); h2.update(msg);      d2 = h2.digest(32)
            if d1 == d2:
                collisions += 1
        else:
            seen[sig] = msg

        if (i + 1) % 100 == 0:
            print(f" Processed {i+1}/{attempts}...")

    elapsed = time.time() - t0
    print(f" Time:           {elapsed:.1f} seconds")
    print(f" Collisions found: {collisions}")
    print(f" Result:    {'ROBUST  ✓' if collisions == 0 else 'WEAK    ✗'}")
    return collisions

```

```

# =====
# TEST 10 — SACRED PAIR VERIFICATION
# =====

```

```

def test_sacred_pair():
    print("\n" + "="*65)
    print("TEST 10: SACRED PAIR (K1 ↔ K2) VERIFICATION")
    print("="*65)

    errors = 0
    for x in range(256):
        sp = sacred_pair(x)
        if sacred_pair(sp) != x: errors += 1
        if (x ^ sp) != 0xFF: errors += 1

    print(f" Double inversion  sacred_pair(sacred_pair(x)) == x : {'PASS ✓' if errors==0 else 'FAIL ✗'}")
    print(f" XOR property      x XOR sacred_pair(x) == 0xFF :   {'PASS ✓' if errors==0 else 'FAIL ✗'}")

    print(f"\n Sample values:")
    print(f" {'x':>5}  -> {'sacred_pair':>11} XOR")
    print(f" {'-'*38}")
    for x in [0, 1, 127, 128, 254, 255]:
        sp = sacred_pair(x)
        print(f" {x:>3} (0x{x:02x}) -> {sp:>3} (0x{sp:02x})  0x{x^sp:02x}")

    print(f"\n Structural note:")
    print(f" 256-byte input -> 255 pyramid anchors (i=0..254).")
    print(f" Sacred pair activates at i>=256 — the K1/K2 boundary.")
    print(f" Reserved structural property for future extensions.")
    return errors == 0

# =====
# TEST 11 — PERFORMANCE
# =====

def test_performance():
    print("\n" + "="*65)
    print("TEST 11: PERFORMANCE (10 runs per size)")
    print("="*65)
    print(f" {'Input':<14} {'Avg ms':>9} Note")
    print(f" {'-'*45}")

    for data, label in [
        (b"x"*256, "256 bytes"),

```

```

(b"x"*512, "512 bytes"),
(b"x"*1024, "1 KB"),
(b"x"*4096, "4 KB"),
]:
    times = []
    for _ in range(10):
        t0 = time.time()
        h = InvertedHash(); h.update(data); h.digest(256)
        times.append((time.time()-t0)*1000)
    avg = sum(times)/len(times)
    print(f" {label:<14} {avg:>7.1f} ms  prototype only")

print(f"\n Note: 24 rounds on 256-byte chunks — pure Python.")
print(f" A C/Rust implementation would be ~100x faster.")

```

```

# =====
# MAIN
# =====

```

```

def main():
    print("\n" + "="*65)
    print(" 512×512 INVERTED HASH — COMPLETE STATISTICAL ANALYSIS")
    print(" Author: Marco Lazcano")
    print("="*65)
    print(" Input:  256 bytes (2048 bits)")
    print(" Output:  256 bytes (2048 bits)")
    print(" Rounds:  24      [Fix B: was 12]")
    print(" Chunk:    256 bytes [Fix A: was 64]")
    print(" Inversion at dimension 257 — K1 ↔ K2 sacred pairs")
    print(" All tests on FULL 256-byte output. No shortcuts.")
    print("="*65)

```

```

basic_ok          = test_basic()
det_ok            = test_determinism(200)
av_mean, av_std   = test_avalanche(150)
entropy_val       = test_entropy(300)
chi2_val          = test_chi_square(600)
corr_avg, corr_max, corr_high = test_correlation(1000)
sbox_prob         = test_sbox_differential()
sac_pct           = test_sac(150)
collisions        = test_collisions(400)
sacred_ok         = test_sacred_pair()
test_performance()

```

```

# — FINAL REPORT —————
def st(ok): return "✓" if ok else "✗"

print("\n" + "="*65)
print(" FINAL REPORT — 512×512 INVERTED HASH (Fix A+B applied)")
print("="*65)
print(f"")

```

512×512 INVERTED HASH — COMPLETE RESULTS (v2)			
Test	Result	Status	
Basic functionality	All distinct	{st(basic_ok)}	
Determinism (200 trials)	0 errors	{st(det_ok)}	
Avalanche (2048 bits)	{av_mean:.3f}% σ={av_std:.3f}%	{ "✓" if abs(av_mean-50)<2 else "△" }	
Shannon entropy	{entropy_val:.6f} b/B	{ "✓" if entropy_val>7.95 else "△" }	
Chi-square uniformity	{chi2_val:.2f}	{ "✓" if chi2_val<293.2 else "△" }	
Pearson corr. avg	{corr_avg:.5f}	{ "✓" if corr_avg<0.10 else "△" }	
Pearson corr. max	{corr_max:.5f}	{ "✓" if corr_max<0.50 else "△" }	
S-box max differential	0.0156 (1.56%)	✓	
SAC deviation > 10%	{sac_pct:.2f}%	{ "✓" if sac_pct<10 else "△" }	
Collision resistance	{collisions} / 400 att.	{ "✓" if collisions==0 else "✗" }	
Sacred pair (K1↔K2)	Verified	{st(sacred_ok)}	
Fix A: chunk 64 -> 256 bytes (full pyramidal block) Fix B: rounds 12 -> 24 (complete diffusion) State: 256 bytes (2048 bits) Output: 256 bytes (2048 bits) — 32,640 pairs analyzed Status: RESEARCH PROTOTYPE — Not for production			


```
print("="*65)
print(" ANALYSIS COMPLETE — 512×512 INVERTED HASH (v2)")
print("="*65)
```

```
if __name__ == "__main__":
    main()
```